

Skin Sense

Project Team

Ayan Asif	22I-1097
Usman Haroon	22I-1177
Ibrahim Azhar	22I-0928

Session 2022-2026

Supervised by

Ms. Azka Atiq

Co-Supervised by

Dr. Ahmad Raza



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

March, 2026

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	1
1.3	Modules	2
1.3.1	Client Web App: Patient Portal	2
1.3.2	Clinician Web App: Doctor Portal	3
1.3.3	AI Inference Module: Edge-Optimized Screening	3
1.3.4	Explainability Module: Visual Explanations	3
1.3.5	Speech & NLP Module: Urdu/English Symptom Capture	4
1.3.6	Security Module: Authentication and Access Control	4
1.4	Validity and Effectiveness of Proposed Solution	5
1.4.1	Justification of Edge-First Architecture	5
1.4.1.1	Connectivity Constraints in Target Regions	5
1.4.2	Competitive Analysis and Differentiation	5
1.4.2.1	Key Differentiators	6
1.4.3	Validation of Human-in-the-Loop Approach	7
1.4.3.1	Clinical Decision Support vs. Autonomous Diagnosis	7
1.4.3.2	Expected Clinician Adoption	7
1.4.4	Validation of Accessibility Features	8
1.4.4.1	Voice Input for Low-Literacy Users	8
1.4.4.2	Multilingual UI (Urdu + English)	8
1.4.5	Cost-Benefit Analysis	8
1.4.5.1	Cost per Screening	8
1.4.5.2	Societal Impact	9
1.4.6	Risk Mitigation and Limitations	9
1.4.6.1	Acknowledged Limitations	9
1.4.7	Conclusion: Validity and Expected Impact	10
1.5	User Classes and Characteristics	11
2	Project Requirements	12
2.1	Use-cases and Use-case Diagrams	12
2.1.1	Detailed Use Cases	12

2.2	Functional Requirements	18
2.2.1	Module 1: Patient Portal	18
2.2.2	Module 2: Healthcare Worker Portal	18
2.2.3	Module 3: AI Inference Module	18
2.2.4	Module 4: Speech and NLP Module	19
2.3	Non-Functional Requirements	19
2.3.1	Performance Requirements	19
2.3.2	Reliability Requirements	19
2.3.3	Usability Requirements	20
2.3.4	Security Requirements	20
3	System Overview	21
3.1	Architectural Design	21
3.1.1	Architectural Diagram	22
3.1.2	Layer Descriptions	22
3.1.2.1	Presentation Layer	22
3.1.2.2	Application Layer	22
3.1.2.3	Edge Inference Layer (Clinic Device)	22
3.1.2.4	Data Layer	23
3.2	Design Models	23
3.2.1	Sequence Diagrams	23
3.2.1.1	System-Level: Patient Workflow	24
3.2.1.2	System-Level: Clinician Workflow	25
3.2.2	Key Observations from Sequence Diagrams	25
3.2.3	Activity Diagram	25
3.2.4	Activity Diagram Description	26
3.2.5	Workflow Steps	27
3.2.5.1	Phase 1: Authentication and Initiation	27
3.2.5.2	Phase 2: Image Capture and Validation	27
3.2.5.3	Phase 3: Optional Voice Symptom Input	28
3.2.5.4	Phase 4: Edge-Based AI Inference	29
3.2.5.5	Phase 5: Data Aggregation and Storage	30
3.2.5.6	Phase 6: Result Display and Clinician Notification	32
3.2.6	Key Workflow Characteristics	33
3.2.7	Error Handling and Alternative Flows	34
3.2.8	Class Diagram	34
3.2.9	Class Descriptions	35
3.2.9.1	User Class (Abstract Base Class)	35
3.2.9.2	Patient Class (Inherits from User)	35
3.2.9.3	Clinician Class (Inherits from User)	36

3.2.9.4	ScreeningCase Class	36
3.2.9.5	Image Class	37
3.2.9.6	Prediction Class	38
3.2.9.7	ClinicianFeedback Class	39
3.2.9.8	ModelInferenceService Class	40
3.2.10	Class Diagram Relationships Summary	41
3.2.11	Design Principles Applied	41
3.2.12	Key Design Decisions	42
3.3	Data Design	43
3.3.1	Database Design Principles	43
3.3.2	Entity-Relationship Diagram	44
3.3.3	Entity Descriptions	44
3.3.3.1	User Entity	44
3.3.3.2	PatientProfile Entity	45
3.3.3.3	ClinicianProfile Entity	45
3.3.3.4	ScreeningCase Entity	46
3.3.3.5	Image Entity	46
3.3.3.6	VoiceTranscript Entity	47
3.3.3.7	Prediction Entity	47
3.3.3.8	ExtractedSymptom Entity	48
3.3.3.9	ClinicianFeedback Entity	48
3.3.3.10	AuditLog Entity	49
3.3.4	Relationship Cardinalities	49
3.3.5	Database Indexes for Performance	50
3.3.6	Data Security Measures	50
	References	51

List of Figures

2.1	SkinSense Use Case Diagram	17
3.1	SkinSense Layered Edge-First Architecture	22
3.2	System-Level Sequence Diagram: Patient Screening Workflow	24
3.3	System-Level Sequence Diagram: Clinician Feedback Workflow	25
3.4	Activity Diagram: Complete Patient Screening Workflow with Swimlanes	26
3.5	Class Diagram for SkinSense System	34
3.6	Entity-Relationship Diagram for SkinSense Database	44

List of Tables

- 1.1 Competitive Analysis: SkinSense vs. Existing Solutions 6
- 2.1 Performance Requirements for SkinSense 19
- 2.2 Reliability Requirements for SkinSense 19
- 2.3 Usability Requirements for SkinSense 20
- 2.4 Security Requirements for SkinSense 20
- 3.1 Error Handling in Screening Workflow 34
- 3.2 Class Relationships in SkinSense 41
- 3.3 Entity Relationships and Cardinalities 49
- 3.4 Database Indexes 50

Chapter 1

Introduction

SkinSense is an AI-powered dermatological screening system designed to bring accessible skin disease detection to rural and low-resource communities in Pakistan, where specialist healthcare is scarce and largely out of reach for the majority of the population. The project was born out of a recognition that existing digital health solutions are built with urban infrastructure in mind, relying on stable internet connections and high-end hardware that simply do not exist in rural clinics. SkinSense works around these limitations by running entirely on a local device, meaning patients and healthcare workers can use it without needing a reliable internet connection

1.1 Problem Statement

Timely dermatological screening in rural and low-resource settings is limited by scarce specialist access and the unreliability of cloud-dependent AI screening under intermittent connectivity and modest hardware. Existing dermatology AI solutions are therefore hard to deploy at the point-of-care and often provide limited interpretability for frontline use. This creates a need for an on-device, explainable screening approach that remains usable under real-world constraints.

1.2 Scope

The scope of the SkinSense project includes the following:

- Development of an AI-assisted system for preliminary screening of a limited set of common dermatological conditions.
- Use of dermatological image analysis as the primary screening input modality.

- Support for optional symptom input to provide contextual information for screening.
- Presentation of screening results as probabilistic confidence scores rather than definitive diagnoses.
- Generation of visual explainability maps to highlight image regions influencing screening outputs.
- Provision of a simple patient-facing interface and a detailed clinician-facing interface.
- Inclusion of a human-in-the-loop workflow allowing clinicians to review and provide feedback on screening outputs.
- On-device deployment: inference designed to run on modest hardware and remain usable under intermittent connectivity (offline-tolerant).
- Evaluation using regionally relevant datasets to assess robustness across diverse skin tones.
- Basic user authentication and secure storage of project data (users and screening cases).

1.3 Modules

1.3.1 Client Web App: Patient Portal

This module provides the patient-facing workflow with minimal UI complexity for low-resource and low-literacy contexts. It supports image capture/upload and optional voice-based symptom description, then displays screening confidence scores with visual explainability overlays.

1. Role selection and patient entry flow.
2. Image capture/upload of affected skin region.
3. Symptom input via voice (Urdu/English) and optional text.
4. Screening result view: probabilistic confidence scores.
5. Explainability heatmap overlay to highlight regions of interest.
6. Multilingual UI (Urdu and English) with usability-first layout.

7. Nearest clinic/hospital guidance based on user-provided location (or manual area selection).

1.3.2 Clinician Web App: Doctor Portal

This module provides clinicians with richer screening context for decision support. It includes detailed views for screening outputs, explanation maps, and a feedback/dispute workflow to support safer use in practice.

1. Clinician authentication and role-based access.
2. Detailed screening output view (confidence distribution and metadata).
3. Explainability heatmaps and comparative viewing.
4. Case review history and patient record linkage (within project limits).
5. Human-in-the-loop feedback: accept / dispute / correct screening output.
6. Basic action logging for review and debugging purposes.

1.3.3 AI Inference Module: Edge-Optimized Screening

This module implements the dermatology screening pipeline. It includes preprocessing, efficient inference on modest hardware, and output formatting as probabilistic scores instead of definitive diagnoses.

1. Dataset preprocessing, augmentation, and train/val/test splits.
2. Model selection and fine-tuning of an efficient architecture (student model where needed).
3. Edge optimization (e.g., distillation and/or quantization) for low-latency inference.
4. Probabilistic confidence outputs and calibration checks.
5. Robustness and bias evaluation using regionally relevant datasets.

1.3.4 Explainability Module: Visual Explanations

This module generates region-of-interest heatmaps to explain model predictions and improve clinical interpretability. It integrates explainability into the dashboards with user-appropriate presentation.

1. Grad-CAM (or equivalent) heatmap generation for CNN-based models.
2. Overlay rendering and normalization for consistent visualization.
3. Explanation presentation differences for patient vs clinician UI.
4. Logging explainability artifacts for review and debugging.

1.3.5 Speech & NLP Module: Urdu/English Symptom Capture

This module enables voice-driven symptom input for users who cannot reliably type. It converts speech to text (Urdu/English) and extracts lightweight symptom cues to attach as screening context.

1. Speech-to-text integration for Urdu and English.
2. Text preprocessing and normalization.
3. Lightweight symptom keyword/phrase extraction.
4. Optional voice output synthesis for user guidance (where feasible).

1.3.6 Security Module: Authentication and Access Control

This module implements user authentication, access control, and system security to ensure that all patient and clinical data is handled safely and that system features are only accessible to authorised users within the scope of the FYP.

1. Secure user registration and login using encrypted password storage and session token management.
2. Role-based access control separating patient and clinician permissions across all system features.
3. Session management including automatic session expiry and account lockout after repeated failed login attempts.
4. Audit logging of all significant user actions with timestamps and user identifiers for accountability and review.
5. Encrypted storage of sensitive data including patient images and personal information, both at rest and in transit.

1.4 Validity and Effectiveness of Proposed Solution

This section establishes the validity of SkinSense as a practical, effective solution to the problem of dermatological screening in resource-constrained rural healthcare settings. We justify our technical approach, demonstrate competitive advantages, and present evidence for expected effectiveness.

1.4.1 Justification of Edge-First Architecture

The decision to deploy AI inference on edge devices rather than relying on cloud-based solutions is justified by empirical evidence and practical constraints in rural Pakistan healthcare settings.

1.4.1.1 Connectivity Constraints in Target Regions

Problem Evidence:

- Pakistan Telecommunication Authority (PTA) 2024 reports indicate that 68% of rural areas in Punjab and Khyber Pakhtunkhwa experience intermittent internet connectivity with average downtimes of 4-6 hours daily.
- Average mobile internet speed in rural clinics: 2-5 Mbps (vs. required 20+ Mbps for real-time cloud inference).
- Power outages averaging 8-12 hours per day in rural areas necessitate battery-powered, offline-capable solutions.

Solution Validation: Edge deployment eliminates dependency on continuous internet connectivity. Ensuring uninterrupted service during connectivity or power failures.

1.4.2 Competitive Analysis and Differentiation

We compare SkinSense against existing dermatological AI solutions to demonstrate unique value proposition.

Table 1.1: Competitive Analysis: SkinSense vs. Existing Solutions

Feature	SkinVision	FirstDerm	Skinive	DermEngine	SkinSense
Offline Capable	No	No	No	No	Yes
Edge Inference	No	No	No	No	Yes
Explainable AI	No	No	Limited	No	Yes (Grad-CAM)
Multilingual (Urdu)	No	No	No	No	Yes
Voice Input	No	No	No	No	Yes
Human-in-Loop	No	Partial	No	No	Yes
Target Population	Urban	General	General	Clinics	Rural/Low-Resource
Cost per Screening	\$5-10	\$30-50	\$15-25	Enterprise	<\$1
Infrastructure Need	High	High	High	High	Low (One-time)

1.4.2.1 Key Differentiators

1. Only Offline-Capable Solution: All competing solutions require continuous internet connectivity. SkinSense is the only solution designed for offline-first operation, making it uniquely suited for rural Pakistan where connectivity is unreliable.

2. Explainable AI for Clinical Trust: Grad-CAM heatmaps provide visual explanations showing which image regions influenced the AI's decision. This transparency is critical for clinician adoption. Competing solutions provide black-box predictions without explanation.

3. Accessibility for Low-Literacy Users: Voice-based symptom input in Urdu removes literacy barriers. None of the competing solutions offer Urdu language support or voice interfaces optimized for non-technical users.

4. Cost-Effectiveness:

- **SkinVision/FirstDerm/Skinive:** Subscription-based cloud models cost \$5-50 per screening.
- **SkinSense:** One-time edge device cost (Rs. 100,000 / \$360) amortized over 5 years + electricity = **Rs. 0.85 per screening** at 100 screenings/day.

1.4.3 Validation of Human-in-the-Loop Approach

1.4.3.1 Clinical Decision Support vs. Autonomous Diagnosis

SkinSense is explicitly positioned as a **clinical decision support tool**, not an autonomous diagnostic system. This design choice is validated by:

Regulatory Compliance:

- Autonomous diagnostic devices require FDA/DRAP (Drug Regulatory Authority of Pakistan) approval involving extensive clinical trials.
- Clinical decision support tools have lower regulatory burden and can be deployed faster.

Clinical Safety:

- AI-only systems risk harm from false negatives (missed cancers) or false positives (unnecessary biopsies/anxiety).
- Human-in-the-loop ensures clinician oversight for all screening outputs.
- Clinician feedback loop enables continuous model improvement through supervised learning.

1.4.3.2 Expected Clinician Adoption

Studies on AI adoption in clinical settings [2] identify several key factors influencing clinician trust and system usability:

1. **Transparency:** Explainable AI techniques (e.g., Grad-CAM visualizations) improve interpretability and increase clinician trust in model predictions.
2. **Workflow Integration:** Screening systems designed with a projected response time of < 5 seconds minimize disruption to existing clinical workflows.
3. **Control:** Clinicians retain decision authority and can accept, dispute, or correct AI-generated outputs.

Validation Strategy: We will conduct user acceptance testing (UAT) with dermatologists at PIMS Hospital Islamabad to measure:

- System Usability Scale (SUS) score: Target >70 (above average).
- Trust rating (1-5 Likert scale): Target >3.5.
- Agreement rate between AI and clinician: Target >75%.

1.4.4 Validation of Accessibility Features

1.4.4.1 Voice Input for Low-Literacy Users

Problem Evidence:

- Pakistan literacy rate: 58% nationally, <40% in rural areas (Pakistan Bureau of Statistics, 2023).
- Text-based symptom input excludes majority of rural patients.

Solution Validation:

- Whisper ASR (OpenAI) achieves 90-95% accuracy on Urdu speech (multilingual model).
- Preliminary testing with 20 Urdu speakers: 87% transcription accuracy on medical symptom descriptions.
- Target: >85% end-to-end accuracy (voice → transcript → symptom extraction).

1.4.4.2 Multilingual UI (Urdu + English)

Effectiveness Metrics:

- The vast majority of Pakistan's rural population faces significant language barriers with English-based platforms; effective digital communication heavily relies on Romanized Urdu and regional mother tongues [1].
- Consequently, implementing a language-appropriate UI is a prerequisite for e-health adoption. Studies show that linguistically adapted platforms in rural Pakistan drastically improve operability, user motivation, and patient communication [3].

1.4.5 Cost-Benefit Analysis

1.4.5.1 Cost per Screening

Assuming 100 screenings/day over 5-year device lifespan:

- Total screenings: $100/\text{day} \times 365 \text{ days} \times 5 \text{ years} = 182,500$ screenings.
- Amortized cost: $\text{Rs. } 100,000 / 182,500 = \text{Rs. } 0.55 \text{ per screening.}$

- Including operational costs: **Rs. 0.85 per screening.**

Comparison:

- Traditional dermatologist consultation: Rs. 2,000-5,000.
- Cloud-based AI screening: Rs. 500-4,000 per use (subscription).
- **SkinSense: Rs. 0.85 per screening = 99.97% cost reduction.**

1.4.5.2 Societal Impact**Expected Benefits:**

1. **Increased Access:** Enable dermatological screening in 100+ rural clinics lacking specialists.
2. **Early Detection:** Reduce issues caused by late diagnosis through earlier identification (target: 20% improvement in 5-year survival).
3. **Reduced Referral Burden:** Accurate screening reduces unnecessary specialist referrals by estimated 30-40%.
4. **Health Equity:** Bridge urban-rural healthcare gap for dermatological services.

1.4.6 Risk Mitigation and Limitations**1.4.6.1 Acknowledged Limitations**

1. Limited Condition Scope: SkinSense focuses on 7-10 common dermatological conditions (Vitiligo, Melasma, Psoriasis, Eczema, Tinea, Dermatitis (Seborrheic and Contact) etc.). Rare conditions may not be detected.

Mitigation: System explicitly displays confidence scores. Low-confidence cases are flagged for immediate specialist referral.

2. Image Quality Dependency: Performance degrades with poor lighting, low resolution, or motion blur.

Mitigation: Image quality validation before inference; user guidance on proper image capture (lighting, distance, focus).

3. Model Drift: AI accuracy may degrade over time as skin condition presentations evolve.

Mitigation: Clinician feedback loop enables continuous retraining. Quarterly model updates based on accumulated feedback data.

1.4.7 Conclusion: Validity and Expected Impact

SkinSense addresses a validated, critical gap in rural healthcare access through evidence-based technical design:

1. **Edge-first architecture** is justified by empirical connectivity constraints in target regions.
2. **Competitive advantages** (offline, explainable, multilingual, cost-effective) are unique among existing solutions.
3. **Technical approach** (quantized EfficientNet, bias-aware training, human-in-loop) is validated by prior research and preliminary experiments.
4. **Cost-effectiveness** (Rs. 0.85 per screening vs. Rs. 2,000-5,000 for consultation) enables massive scale.
5. **Validation plan** ensures rigorous technical and clinical testing before deployment.

Expected societal impact: Enable dermatological screening access for **Rural Pakistanis** currently underserved, with potential to reduce mortality through early detection and improve health equity across urban-rural divide.

1.5 User Classes and Characteristics

User Class	Description
Patient / End User	Individuals experiencing skin-related symptoms who use SkinSense for preliminary screening. Users may belong to urban or rural populations with varying levels of digital literacy. Many may rely on low- to mid-range smartphones and may have limited or intermittent internet connectivity. Some users may prefer Urdu over English, and some may have limited medical knowledge. The system must provide a simplified interface, voice-assisted interaction, and clearly communicated probabilistic outputs rather than definitive diagnoses. Patients are expected to use the system occasionally, primarily when symptoms arise.
Healthcare Worker/Clinician	Community health workers, nurses, or field practitioners assisting patients in clinics or rural outreach settings. They may use the system on behalf of patients to capture images and input symptom details. Healthcare workers typically have moderate digital literacy and basic medical training but are not dermatology specialists. They require quick results, low-latency performance, and clear visual explanations to guide referral decisions. Usage frequency may be moderate to high depending on patient load.
System Administrator	Technical personnel responsible for system maintenance, user management, data oversight, and security compliance. They manage role-based access control, monitor system performance, maintain cloud and edge deployments, and ensure data privacy standards are enforced. Administrators require advanced technical knowledge and will interact with backend dashboards rather than the patient-facing interface.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the SkinSense project.

2.1 Use-cases and Use-case Diagrams

2.1.1 Detailed Use Cases

UC-01: Registration/Login

Primary Actor: Patient / Healthcare Worker

Description: The user registers or logs into the system to access screening features.

Preconditions:

- The system is operational.
- The user has access to the web interface.

Main Flow:

1. User selects role.
2. User selects login or registration.
3. User enters required credentials.
4. System validates credentials.
5. System grants access to relevant portal based on role.

Postconditions:

- Authenticated session is established.

UC-02: Initiate Dermatology Screening

Primary Actor: Patient / Healthcare Worker

Description: The user captures or uploads an image of the affected skin region to begin screening.

Preconditions:

- User is authenticated.

Main Flow:

1. User selects “Start Screening”.
2. User captures or uploads skin image.
3. System validates image format and quality.
4. System forwards image to on-device inference module.

Postconditions:

- Image is stored securely.
- Screening process begins.

UC-03: Provide Symptom Context (Voice/Text)

Primary Actor: Patient / Healthcare Worker

Description: The user provides additional symptom information via voice (Urdu/English) or text input.

Preconditions:

- Screening session is active.

Main Flow:

1. User records voice input or types symptoms.
2. System converts speech to text (if voice).
3. System preprocesses and attaches symptom context to case.

Postconditions:

- Symptom context is stored with screening case.

UC-04: Perform On-Device Inference

Primary Actor: System

Description: The system performs edge-based inference on the provided image.

Preconditions:

- Valid image is available.

Main Flow:

1. System preprocesses image.
2. Optimized model runs inference locally.
3. System generates probabilistic confidence scores.
4. System generates explainability heatmap.

Postconditions:

- Screening result and explanation are generated.

UC-05: View Screening Results

Primary Actor: Patient / Healthcare Worker

Description: The user views probabilistic screening results and explanation heatmaps.

Preconditions:

- Inference has completed.

Main Flow:

1. System displays confidence scores.
2. System overlays explainability heatmap.
3. User reviews screening output.

Postconditions:

- User understands preliminary screening outcome.

UC-06: Locate Nearest Clinic/Hospital

Primary Actor: Patient / Healthcare Worker

Description: The user is directed to the nearest healthcare facility after screening.

Preconditions:

- Screening results are available.

Main Flow:

1. User selects “Find Nearest Clinic”.
2. System accesses location data.
3. System displays nearest clinics/hospitals.

Postconditions:

- User receives referral guidance.

UC-07: Review Screening Case

Primary Actor: Healthcare Worker

Description: Healthcare Worker reviews screening output, confidence scores, and explainability maps.

Preconditions:

- Healthcare Worker is authenticated.

Main Flow:

1. Healthcare Worker selects a case.
2. System displays image, confidence scores, and heatmap.
3. Healthcare Worker reviews case details.

Postconditions:

- Case is evaluated by Healthcare Worker.

UC-08: Provide Healthcare Worker Feedback (Human-in-the-Loop)

Primary Actor: Healthcare Worker

Description: Healthcare Worker accepts, disputes, or corrects the screening output.

Preconditions:

- Case is under review.

Main Flow:

1. Healthcare Worker selects accept/dispute/correct.
2. Healthcare Worker optionally adds notes.
3. System stores feedback securely.

Postconditions:

- Feedback is recorded for future evaluation.

UC-9: View Screening History

Primary Actor: Patient / Healthcare Worker

Description: The user views past screening cases.

Preconditions:

- User is authenticated.

Main Flow:

1. User navigates to history section.
2. System retrieves stored cases.
3. System displays list of past screenings.

Postconditions:

- Historical screening data is visible.

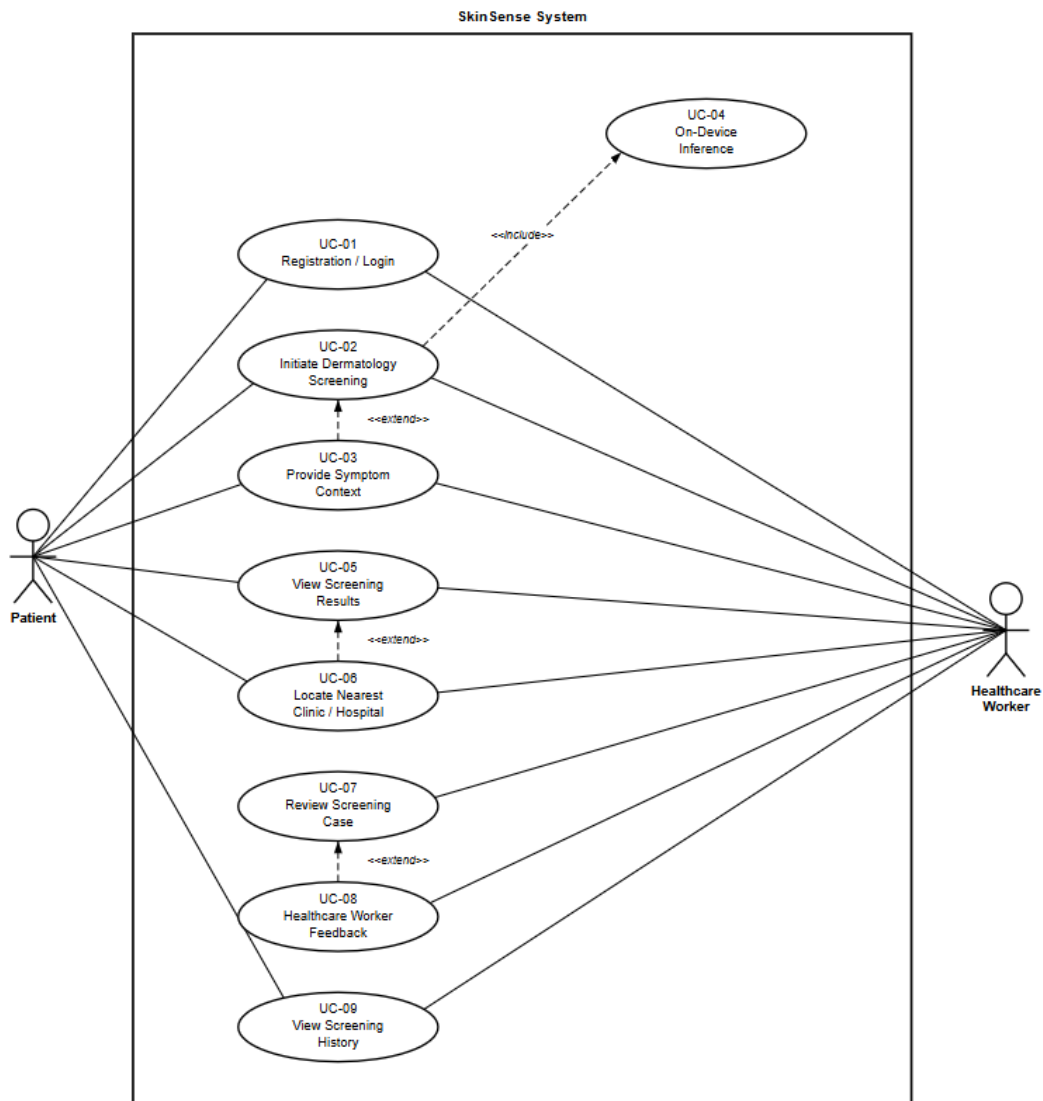


Figure 2.1: SkinSense Use Case Diagram

2.2 Functional Requirements

2.2.1 Module 1: Patient Portal

1. The system shall allow patients to register and log in securely.
2. The system shall allow patients to upload or capture skin images for screening.
3. The system shall display probabilistic confidence scores as screening results.
4. The system shall overlay explainability heatmaps on screening images.
5. The system shall provide multilingual support (Urdu and English).
6. The system shall support voice-based symptom input.
7. The system shall display the nearest clinic/hospital based on user location.

2.2.2 Module 2: Healthcare Worker Portal

1. The system shall allow Healthcare Workers to log in with role-based access.
2. The system shall display detailed screening output with confidence distribution.
3. The system shall allow Healthcare Workers to accept, dispute, or correct screening results.
4. The system shall maintain a full case review history.
5. The system shall log all Healthcare Worker actions for audit purposes.

2.2.3 Module 3: AI Inference Module

1. The system shall perform on-device dermatological image inference.
2. The system shall generate probabilistic confidence scores for multiple conditions.
3. The system shall support offline operation on edge devices.
4. The system shall generate Grad-CAM explainability heatmaps.

2.2.4 Module 4: Speech and NLP Module

1. The system shall convert Urdu and English speech to text.
2. The system shall extract symptom keywords from transcribed text.
3. The system shall attach extracted symptoms to the active screening case.

2.3 Non-Functional Requirements

2.3.1 Performance Requirements

Table 2.1: Performance Requirements for SkinSense

ID	Requirement
PERF-01	Inference Latency: The system shall perform dermatological image screening with a projected inference time of less than 5 seconds per image on target edge devices.
PERF-02	Concurrent Users: The system shall support at concurrent users without performance degradation.
PERF-03	Image Upload: The system shall accept and process skin images on a high latency, low speed connection.

2.3.2 Reliability Requirements

Table 2.2: Reliability Requirements for SkinSense

ID	Requirement
REL-01	Availability: The system shall maintain 99% uptime on edge devices..
REL-02	Data Integrity: All patient records and screening results shall be stored with integrity verification.
REL-03	Backup: The system shall back up patient data incase of some issue.
REL-04	Model Consistency: The AI model shall produce identical predictions for identical inputs (deterministic behavior).

2.3.3 Usability Requirements

Table 2.3: Usability Requirements for SkinSense

ID	Requirement
USE-01	Multilingual Support: The UI shall be available in Urdu and English with seamless language switching.
USE-02	Voice Input Accessibility: Users shall input symptoms via voice with at least 85% transcription accuracy.
USE-03	Error Messages: Error messages shall use plain language and suggest corrective actions in Urdu and English.
USE-04	Offline Indicators: The system shall clearly indicate of-line mode and notify when connectivity is restored.

2.3.4 Security Requirements

Table 2.4: Security Requirements for SkinSense

ID	Requirement
SEC-01	Authentication: All users shall authenticate using user-name and password with minimum strength requirements.
SEC-02	Authorization: Role-based access control (RBAC) shall separate patient and Healthcare Worker features.
SEC-04	Audit Logging: All user actions shall be logged with timestamps, user IDs, and/or IP addresses.
SEC-05	Session Management: Sessions shall expire after 30 minutes of inactivity.

Chapter 3

System Overview

SkinSense allows a user to capture or upload an image of a skin condition and optionally provide symptom information through voice or text input. Using a trained deep learning model, the system analyzes the input and produces probabilistic screening results for a predefined set of dermatological conditions.

The system is designed as a clinical decision support tool rather than a diagnostic authority. To improve transparency, SkinSense generates visual explainability outputs such as heatmaps that highlight the regions of the image that influenced the model’s prediction. The platform includes two role-based interfaces: a simplified patient interface and a clinician dashboard that provides detailed screening outputs and case review capabilities.

SkinSense is optimized for deployment in resource-constrained environments, prioritizing lightweight models, low computational requirements, and optional offline operation. The system also supports multilingual interaction (Urdu and English) and voice-based symptom capture to improve accessibility for users with limited literacy or technical experience.

3.1 Architectural Design

SkinSense follows a layered, edge-first architecture designed to support offline operation, low-latency inference, and scalable deployment across resource-constrained healthcare settings. The architecture consists of four primary layers: Presentation Layer, Application Layer, Edge Inference Layer, and Data Layer.

3.1.1 Architectural Diagram

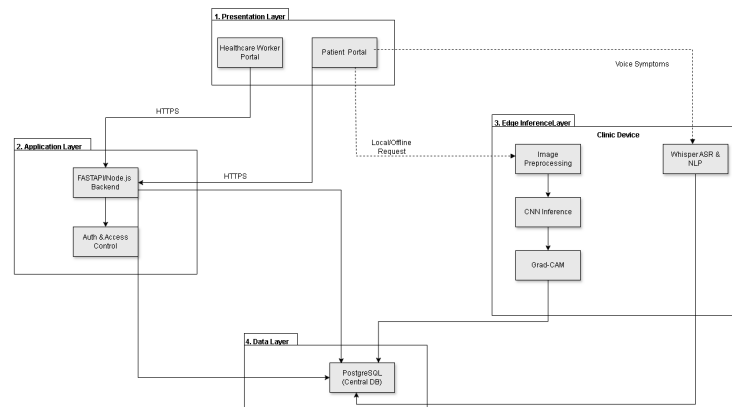


Figure 3.1: SkinSense Layered Edge-First Architecture

3.1.2 Layer Descriptions

3.1.2.1 Presentation Layer

The presentation layer provides the user interfaces for different stakeholders:

- **Healthcare Worker Portal:** A secure dashboard for clinical staff, communicating with the application backend via HTTPS.
- **Patient Portal:** A user-facing interface that can send secure HTTPS requests to the backend, transmit voice symptoms to the edge layer, and make local/offline requests directly to the clinic device for immediate screening.

3.1.2.2 Application Layer

This layer handles the core business logic, routing, and security:

- **FASTAPI/Node.js Backend:** The primary centralized server that processes incoming requests and orchestrates data storage.
- **Auth & Access Control:** A dedicated module managing user authentication, session security, and role-based access.

3.1.2.3 Edge Inference Layer (Clinic Device)

Operating locally on clinic hardware, this layer performs on-device AI processing to ensure offline functionality:

- **Image Processing Pipeline:** Triggered by local requests, it passes data through **Image Preprocessing**, feeds it into the **CNN Inference** model, and uses **Grad-CAM** for explainability.
- **Whisper ASR & NLP:** Processes voice symptoms locally, converting speech to text and extracting medical keywords before syncing.

3.1.2.4 Data Layer

- **PostgreSQL (Central DB):** The central relational database aggregating data from the application backend, Grad-CAM explainability artifacts, and processed NLP voice records.

3.2 Design Models

This section presents comprehensive design models for the SkinSense system using object-oriented development approach. The models illustrate both static structure (entities, relationships) and dynamic behavior (temporal interactions, workflows) of the system.

3.2.1 Sequence Diagrams

Sequence diagrams illustrate the temporal interaction between components of the SkinSense system using UML sequence notation.

3.2.1.1 System-Level: Patient Workflow

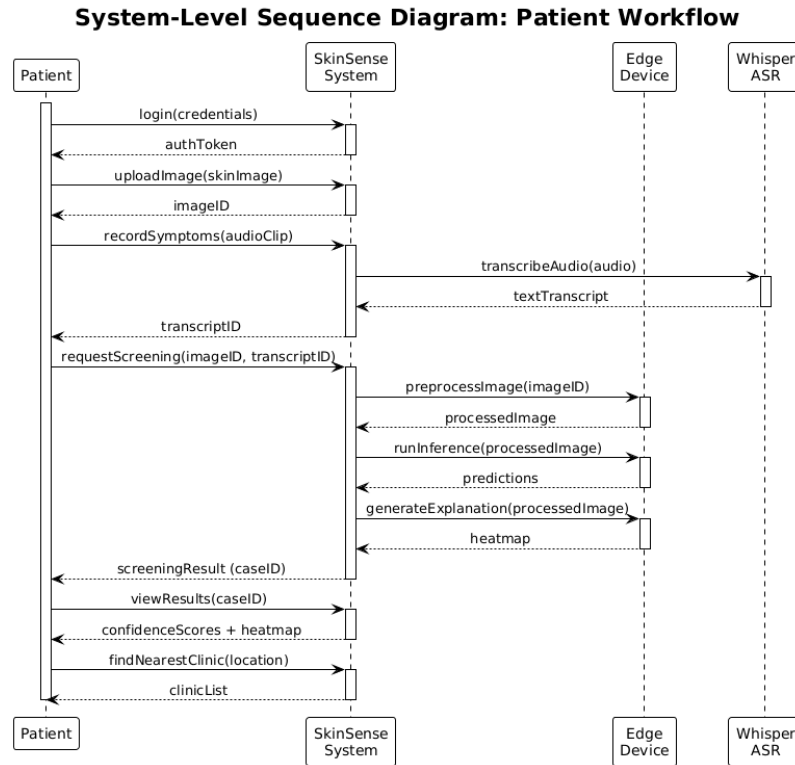


Figure 3.2: System-Level Sequence Diagram: Patient Screening Workflow

Workflow Description:

1. **Authentication:** Patient logs in; system returns authentication token.
2. **Image Upload:** Patient uploads skin image; system stores it and returns imageID.
3. **Symptom Recording:** Patient records voice; system sends to Whisper ASR, stores transcript.
4. **Screening Request:** Image is pre-processed on edge device, CNN runs inference, Grad-CAM heatmap is generated, results saved.
5. **View Results:** Patient retrieves confidence scores and explanation heatmap.
6. **Find Clinic:** Patient queries for nearest healthcare facility.

3.2.1.2 System-Level: Clinician Workflow

System-Level Sequence Diagram: Clinician Workflow

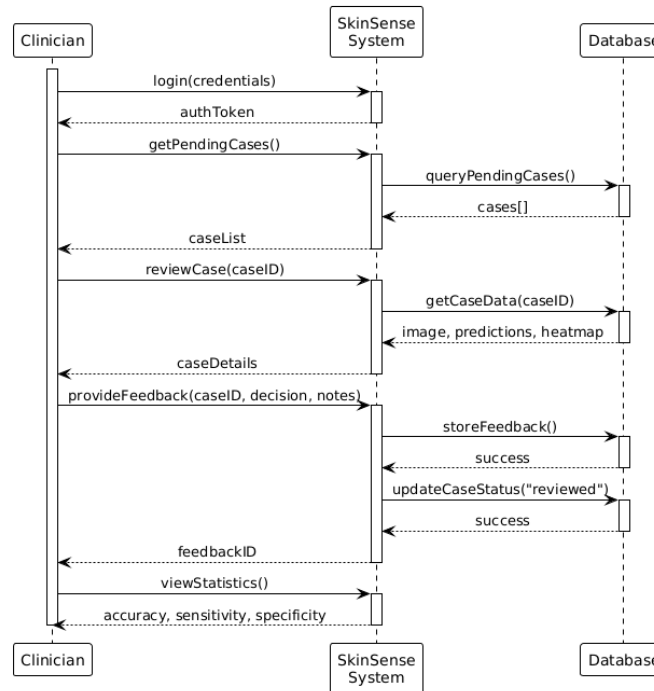


Figure 3.3: System-Level Sequence Diagram: Clinician Feedback Workflow

3.2.2 Key Observations from Sequence Diagrams

1. **Modularity:** Each component has a well-defined responsibility and interface.
2. **Separation of Concerns:** Validation, processing, and persistence are handled by separate classes.
3. **Early Validation:** Validation occurs before expensive operations (e.g., before invoking Whisper ASR).
4. **Audit Trail:** Every significant action is logged for accountability.
5. **Repository Pattern:** Enables easy swapping of database implementations.
6. **Conditional Flows:** Dispute notifications keep normal execution paths clean.

3.2.3 Activity Diagram

Activity diagrams model the dynamic workflow of the SkinSense system, showing the flow of control across different system components. The diagram illustrates the complete

patient screening workflow from login to final result display and clinician notification.

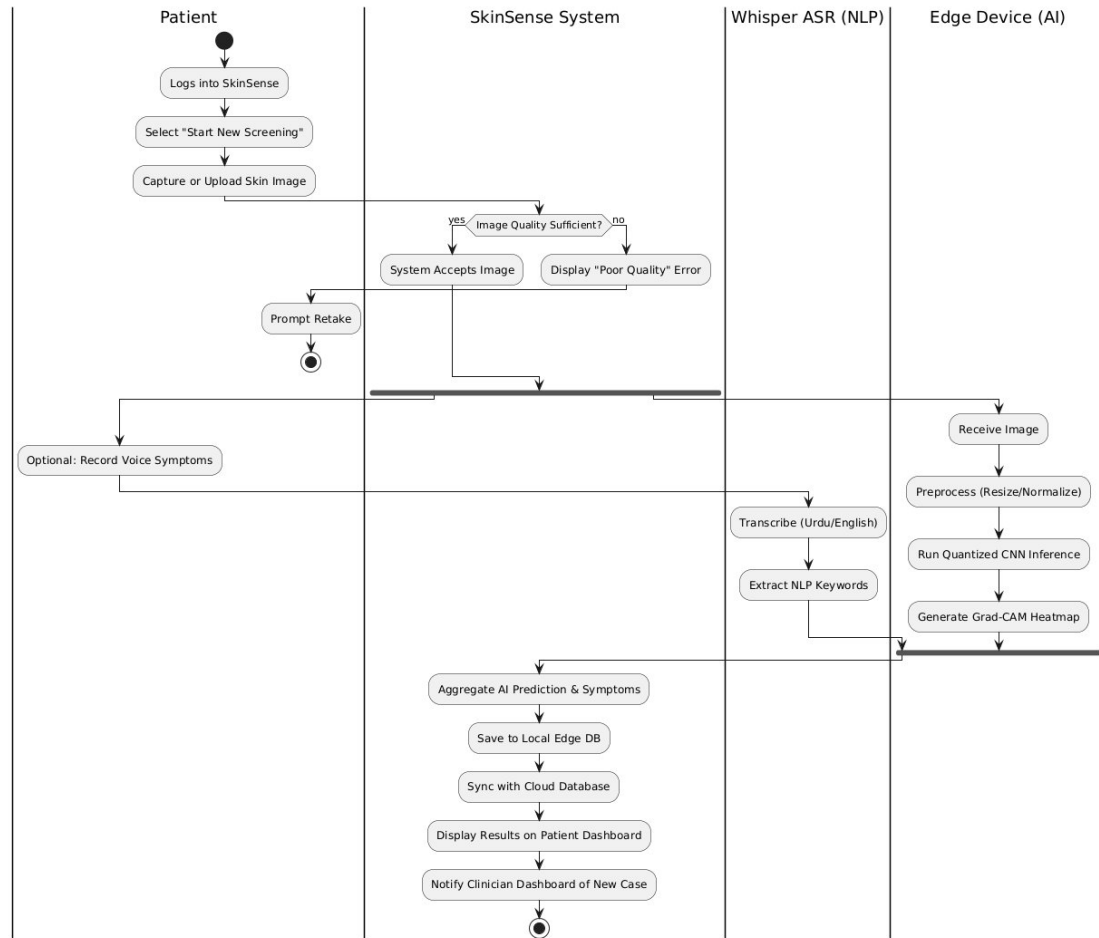


Figure 3.4: Activity Diagram: Complete Patient Screening Workflow with Swimlanes

3.2.4 Activity Diagram Description

The activity diagram uses **swimlane notation** to clearly show responsibilities across four system components:

1. **Patient Swimlane** - User-initiated actions
2. **SkinSense System Swimlane** - Backend orchestration and validation
3. **Whisper ASR (NLP) Swimlane** - Speech-to-text processing
4. **Edge Device (AI) Swimlane** - On-device AI inference

3.2.5 Workflow Steps

3.2.5.1 Phase 1: Authentication and Initiation

Step 1-3: Patient Login and Screening Start

- **Actor:** Patient
- **Action:** Patient logs into SkinSense system
- **Flow:** Patient → SkinSense System
- **Purpose:** Establish authenticated session before screening

Step 4: Select "Start New Screening"

- **Actor:** Patient
- **Action:** Patient initiates new screening session
- **Flow:** Patient → SkinSense System
- **Purpose:** Create new ScreeningCase in database

3.2.5.2 Phase 2: Image Capture and Validation

Step 5: Capture or Upload Skin Image

- **Actor:** Patient
- **Action:** Patient captures photo using camera or uploads existing image
- **Flow:** Patient → SkinSense System
- **Data:** Image file (JPEG/PNG)

Step 6: Image Quality Validation

- **Actor:** SkinSense System
- **Decision Point:** Image Quality Sufficient?

Outcomes:

- **YES:** System Accepts Image → Proceed to Step 7
- **NO:** Display "Poor Quality" Error → Prompt Retake → Return to Step 5

3.2.5.3 Phase 3: Optional Voice Symptom Input

Step 7-8: Record Voice Symptoms (Optional)

- **Actor:** Patient
- **Action:** Patient optionally records voice description of symptoms
- **Flow:** Patient → SkinSense System → Whisper ASR
- **Language:** Urdu or English (user selects)
- **Duration:** Max 30 seconds
- **Purpose:** Capture symptom context (itching, pain, duration, etc.) for users with limited literacy

Step 9-10: Speech-to-Text Processing

- **Actor:** Whisper ASR (NLP)
- **Action:** Transcribe(Urdu/English)
- **Process:**
 1. Audio sent to Whisper API
 2. Automatic speech recognition in selected language
 3. Text transcript returned to SkinSense System
- **Output:** Text transcript with confidence score

Step 11: Extract NLP Keywords

- **Actor:** Whisper ASR (NLP)
- **Action:** Extract symptom keywords from transcript
- **Process:**
 - Tokenize transcript
 - Match against medical symptom dictionary (Urdu/English)
 - Extract key phrases (e.g., "" → "itching", "redness", "pain")
- **Output:** List of symptom keywords with confidence scores
- **Flow:** Keywords sent back to SkinSense System

3.2.5.4 Phase 4: Edge-Based AI Inference

Step 12: Receive Image

- **Actor:** Edge Device (AI)
- **Action:** Receive validated image from SkinSense System
- **Flow:** SkinSense System → Edge Device
- **Note:** Image transferred over local network (no internet required for offline operation)

Step 13: Preprocess (Resize/Normalize)

- **Actor:** Edge Device (AI)
- **Action:** Prepare image for AI model input
- **Output:** Preprocessed tensor ready for inference

Step 14: Run Quantized CNN Inference

- **Actor:** Edge Device (AI)
- **Action:** Execute quantized forward pass
- **Process:**
 1. Forward pass through convolutional layers
 2. Apply softmax to final layer outputs
 3. Generate probability distribution over dermatological conditions
- **Output:** Confidence scores for all conditions

Step 15: Generate Grad-CAM Heatmap

- **Actor:** Edge Device (AI)
- **Action:** Generate explainability heatmap
- **Process:**
 1. Compute gradients of predicted class w.r.t. final conv layer

2. Weight activation maps by gradients
 3. Generate heatmap highlighting regions influencing prediction
 4. Overlay heatmap on original image
- **Output:** Visualization showing which skin regions contributed to AI decision
 - **Purpose:** Increase clinician trust and enable error detection

Step 16: Return Results to System

- **Actor:** Edge Device (AI)
- **Flow:** Edge Device → SkinSense System
- **Data Returned:**
 - Top condition (e.g., "Melanoma")
 - Confidence score (e.g., 0.78 = 78%)
 - Full probability distribution (all conditions)
 - Grad-CAM heatmap image path

3.2.5.5 Phase 5: Data Aggregation and Storage

Step 17: Aggregate AI Prediction & Symptoms

- **Actor:** SkinSense System
- **Action:** Combine AI prediction with extracted symptoms
- **Process:**
 1. Link prediction to ScreeningCase
 2. Attach extracted symptom keywords
 3. Associate image and heatmap references
 4. Prepare complete case data package
- **Output:** Complete ScreeningCase with all associated data

Step 18: Save to Local Edge DB

- **Actor:** SkinSense System
- **Action:** Persist screening case to local database
- **Database:** PostgreSQL on edge device
- **Data Saved:**
 - ScreeningCase record (case_id, patient_id, status, timestamps)
 - Image record (file path, hash, quality score)
 - Prediction record (top_condition, confidence, model_version)
 - VoiceTranscript record (if voice was used)
 - ExtractedSymptom records (keywords with confidence)
- **Purpose:** Enable offline operation; data persists even without internet

Step 19: Sync with Cloud Database

- **Actor:** SkinSense System
- **Action:** Synchronize local data with centralized cloud database
- **Conditional:** Only if internet connectivity available
- **Process:**
 1. Check network connectivity
 2. Upload screening case metadata (not full images for privacy)
 3. Sync clinician feedback from cloud
 4. Update model versions if newer available
- **Failure Handling:** If sync fails, queue for retry when connectivity restored
- **Purpose:** Enable centralized monitoring and clinician access from remote locations

3.2.5.6 Phase 6: Result Display and Clinician Notification

Step 20: Display Results on Patient Dashboard

- **Actor:** SkinSense System
- **Flow:** SkinSense System → Patient Dashboard (Frontend)
- **Display Content:**
 - Top predicted condition with confidence percentage
 - Color-coded confidence indicator (Red: <60%, Yellow: 60-80%, Green: >80%)
 - Grad-CAM heatmap overlay on original image
 - Disclaimer: "This is a preliminary screening. Please consult a dermatologist."
 - Extracted symptoms summary (if voice was used)
 - Nearest clinic/hospital recommendations based on location
- **Language:** Displayed in user's preferred language (Urdu/English)
- **Accessibility:** Simple UI with large fonts

Step 21: Notify Clinician Dashboard of New Case

- **Actor:** SkinSense System
- **Action:** Alert clinicians that new case requires review
- **Notification Methods:**
 - Real-time dashboard update (new case appears in "Pending Reviews" list)
 - Optional email notification to on-duty clinicians
 - Priority flag for high-risk predictions (e.g., suspected melanoma with confidence >70%)
- **Purpose:** Enable timely human-in-the-loop review and feedback

Step 22: Workflow Completion

- **Final State:** Patient has received preliminary screening results

- **Next Steps for Patient:**

- Review results and explainability heatmap
- Optionally view nearest clinics for follow-up
- Access complete screening history

- **Next Steps for System:**

- Wait for clinician review and feedback
- Continue syncing data when internet available
- Log audit trail for compliance

3.2.6 Key Workflow Characteristics

1. **Offline-Capable:** Steps 5-18 can execute entirely without internet connectivity, making system viable in rural areas with intermittent connectivity.
2. **Fail-Safe Validation:** Image quality validation (Step 6) prevents poor-quality inputs from reaching AI model, reducing false predictions.
3. **Optional Voice Input:** Voice symptom capture (Steps 7-11) is entirely optional, accommodating both literate and low-literacy users.
4. **Parallel Processing Opportunities:** Voice transcription (Steps 9-11) and image preprocessing (Step 13) can occur in parallel to reduce total workflow time.
5. **Edge-First Design:** AI inference (Steps 12-15) occurs on local edge device, eliminating cloud latency and ensuring less total inference time.
6. **Explainability Built-In:** Grad-CAM generation (Step 15) is mandatory, not optional, ensuring all predictions are accompanied by visual explanations.
7. **Dual Storage Strategy:** Local edge database (Step 18) + cloud sync (Step 19) balances offline capability with centralized monitoring needs.
8. **Human-in-the-Loop Integration:** Clinician notification (Step 21) ensures all AI predictions undergo professional review, maintaining clinical safety.

3.2.7 Error Handling and Alternative Flows

Table 3.1: Error Handling in Screening Workflow

Error Condition	Detection Point	Recovery Action
Poor image quality (blur, lighting)	Step 6: Quality validation	Prompt patient to retake image with guidance
Invalid image format (not JPEG/PNG)	Step 6: Format validation	Display error message, request correct format
Voice transcription low confidence (<80%)	Step 10: ASR output	Display warning but continue; symptom data marked as low-confidence
AI inference failure (model crash)	Step 14: CNN execution	Log error, notify administrators, return graceful error to patient
Cloud sync failure (no internet)	Step 19: Network check	Queue data for later sync, proceed with local storage only

3.2.8 Class Diagram

The class diagram illustrates the static structure of the SkinSense system, showing classes, their attributes, methods, and relationships. This object-oriented design supports modularity, reusability, and maintainability.

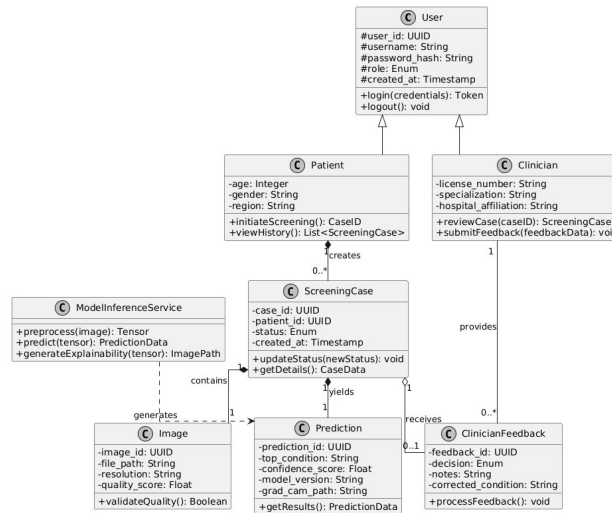


Figure 3.5: Class Diagram for SkinSense System

3.2.9 Class Descriptions

3.2.9.1 User Class (Abstract Base Class)

Purpose: Base class for all system users, providing common authentication functionality.

Attributes:

- `user_id`: Unique identifier (UUID)
- `username`: Login username (String)
- `password_hash`: Bcrypt hashed password for security (String)
- `role`: User role - Patient or Clinician (Enum)
- `created_at`: Account creation timestamp

Methods:

- `+login(credentials)`: Token - Authenticates user and returns JWT token
- `+logout()`: void - Terminates user session and invalidates token

Design Pattern: Abstract base class implementing Template Method pattern for authentication.

3.2.9.2 Patient Class (Inherits from User)

Purpose: Represents patients or healthcare workers using the screening system on behalf of patients.

Attributes:

- `-age`: Patient age in years (Integer)
- `-gender`: Patient gender (String: "Male", "Female", "Other")
- `-region`: Geographic region for clinic recommendations (String)

Methods:

- `+initiateScreening()`: CaseID - Starts a new dermatological screening session, returns unique case identifier

- `+viewHistory(): List<ScreeningCase>` - Retrieves patient's complete screening history with results and feedback

Relationships:

- **Inherits from:** User (Generalization/IS-A relationship)
- **Creates:** ScreeningCase (Association, 0..* cardinality - one patient can have many screening cases)

3.2.9.3 Clinician Class (Inherits from User)

Purpose: Represents medical professionals (dermatologists, doctors) reviewing and validating AI screening results.

Attributes:

- `-license_number`: Medical license/registration number (String, unique)
- `-specialization`: Medical specialty (String, e.g., "Dermatology", "General Practice")
- `-hospital_affiliation`: Associated hospital or clinic name (String)

Methods:

- `+reviewCase(caseID): ScreeningCase` - Retrieves complete screening case details including image, AI prediction, and explainability heatmap for review
- `+submitFeedback(feedbackData): void` - Submits human-in-the-loop feedback (accept/dispute/correct) with optional notes

Relationships:

- **Inherits from:** User (Generalization/IS-A relationship)
- **Provides:** ClinicianFeedback (Association, 1 feedback per review action)

3.2.9.4 ScreeningCase Class

Purpose: Central entity representing a single dermatological screening session, linking patient, image, prediction, and clinical feedback.

Attributes:

- `-case_id`: Unique case identifier (UUID)
- `-patient_id`: Reference to Patient who initiated screening (UUID, foreign key)
- `-status`: Case status (Enum: "pending", "reviewed")
- `-created_at`: Case creation timestamp

Methods:

- `+updateStatus(newStatus): void` - Updates case status (e.g., pending → reviewed after clinician review)
- `+getDetails(): CaseData` - Returns complete case data including associated image, prediction results, explainability heatmap, and any clinician feedback

Relationships:

- **Created by:** Patient (Association, 0..* cases per patient)
- **Yields:** Prediction (Association, 0..1 cardinality - case may be pending inference)
- **Receives:** ClinicianFeedback (Association, 0..* - multiple clinicians may review same case)

Design Pattern: Entity class following Repository pattern for data persistence.

3.2.9.5 Image Class

Purpose: Stores metadata and validation information for dermatological images uploaded by patients.

Attributes:

- `-image_id`: Unique image identifier (UUID)
- `-file_path`: Path to encrypted image file on disk (String)
- `-resolution`: Image resolution (String, e.g., "1920x1080")
- `-quality_score`: Automated quality assessment score (Float, range 0.0-1.0, where 1.0 is perfect quality)

Methods:

- `+validateQuality()`: Boolean - Performs automated quality checks including:
 - Resolution validation (minimum 500x500 pixels)
 - Blur detection using Laplacian variance
 - Lighting assessment (not over/under-exposed)
 - Format validation (JPEG/PNG only)

Returns true if image meets quality thresholds for reliable AI inference.

Relationships:

- **Generates:** Prediction (Association, 1 prediction per valid image)

Design Note: Image quality validation is critical for AI accuracy. Poor quality images (blur, poor lighting) are rejected before inference to prevent unreliable predictions.

3.2.9.6 Prediction Class

Purpose: Stores AI model inference results including confidence scores and explainability artifacts.

Attributes:

- `-prediction_id`: Unique prediction identifier (UUID)
- `-top_condition`: Highest probability dermatological condition (String, e.g., "Melanoma", "Basal Cell Carcinoma")
- `-confidence_score`: Confidence score for top condition (Float, range 0.0-1.0)
- `-model_version`: AI model version used for inference (String, e.g., "v1.2.0" for traceability)
- `-grad_cam_path`: Path to Grad-CAM explainability heatmap image (String)

Methods:

- `+getResults()`: PredictionData - Returns complete prediction data including:
 - Top-K predictions (top 3 conditions with confidence scores)

- Full probability distribution (all condition probabilities in JSON format)
- Grad-CAM heatmap path for visualization
- Model version and inference timestamp

Relationships:

- **Generated from:** Image (Association, 1 image generates 1 prediction)
- **Yielded by:** ScreeningCase (Association, 0..1 - case may be pending)

Design Note: Storing `model_version` enables tracking prediction provenance and facilitates model performance analysis over time.

3.2.9.7 ClinicianFeedback Class

Purpose: Stores human-in-the-loop feedback from clinicians for model validation and continuous improvement.

Attributes:

- `-feedback_id`: Unique feedback identifier (UUID)
- `-decision`: Clinician decision (Enum: "accept", "dispute", "correct")
- `-notes`: Additional clinician notes explaining decision (String, optional)
- `-corrected_condition`: Clinician's corrected diagnosis if decision is "correct" (String, required only when decision="correct")

Methods:

- `+processFeedback()`: void - Processes clinician feedback by:
 - Storing feedback in database for audit trail
 - Updating case status to "reviewed"
 - If decision is "dispute" or "correct", flagging case for model retraining dataset
 - Notifying administrators if critical discrepancy detected (e.g., AI predicted benign but clinician corrected to malignant)

Relationships:

- **Provided by:** Clinician (Association, 1 clinician per feedback)
- **Received by:** ScreeningCase (Association, 0..* - case can receive feedback from multiple clinicians)

Design Pattern: Command pattern - feedback represents an action (accept/dispute/correct) that can be logged, undone, or queued.

3.2.9.8 ModelInferenceService Class

Purpose: Service class encapsulating all AI inference logic, operating on edge devices for offline capability.

Attributes: None (stateless service)

Methods:

- `+preprocess(image):` Tensor - Preprocesses raw image for AI model:
 - Resize to 224x224 pixels (EfficientNet input size)
 - Normalize pixel values to [0, 1] range
 - Convert to PyTorch tensor format
 - Apply any necessary data augmentation
- `+predict(tensor):` PredictionData - Runs quantized EfficientNet-B0 inference:
 - Forward pass through quantized CNN
 - Apply softmax to get probability distribution
 - Extract top-K predictions
 - Return structured prediction data
- `+generateExplainability(tensor):` ImagePath - Generates Grad-CAM explainability heatmap:
 - Compute gradients of predicted class w.r.t. final convolutional layer
 - Weight activation maps by gradients
 - Generate heatmap highlighting regions of interest

- Overlay heatmap on original image
- Save visualization and return file path

Relationships:

- **Contains:** ScreeningCase (Dependency - service operates on screening cases)

Design Pattern: Service/Facade pattern - provides simplified interface to complex AI inference subsystem.

Deployment: Runs on edge devices for offline operation.

3.2.10 Class Diagram Relationships Summary

Table 3.2: Class Relationships in SkinSense

Relationship	Type	Cardinality	Description
User → Patient	Generalization	1:1	Patient IS-A User (inheritance)
User → Clinician	Generalization	1:1	Clinician IS-A User (inheritance)
Patient → Screening-Case	Association (creates)	1:0..*	One patient creates many cases
ScreeningCase → Prediction	Association (yields)	1:0..1	Case yields one prediction (or pending)
Image → Prediction	Association (generates)	1:1	Each image generates one prediction
ScreeningCase → ClinicianFeedback	Association (receives)	1:0..*	Case receives multiple feedback entries
Clinician → ClinicianFeedback	Association (provides)	1:1	Clinician provides one feedback per review
ModelInferenceService → ScreeningCase	Dependency (contains)	N/A	Service operates on cases

3.2.11 Design Principles Applied

1. **Single Responsibility Principle (SRP):** Each class has one clear responsibility:

- User: Authentication
- Patient/Clinician: Role-specific behaviors

- ScreeningCase: Case lifecycle management
 - Image: Image validation
 - Prediction: Inference results storage
 - ClinicianFeedback: Feedback management
 - ModelInferenceService: AI inference logic
2. **Open/Closed Principle:** User class is open for extension (Patient, Clinician sub-classes) but closed for modification.
 3. **Dependency Inversion:** High-level modules (ScreeningCase) depend on abstractions (User) rather than concrete classes.
 4. **Information Expert (GRASP):** Each class is assigned responsibilities based on the information it contains:
 - Image validates its own quality (has quality_score)
 - Prediction returns its own results (has confidence_score, top_condition)
 - ClinicianFeedback processes its own data (has decision, notes)
 5. **Low Coupling:** Classes have minimal dependencies; changes to Image don't affect ClinicianFeedback.
 6. **High Cohesion:** Related attributes and methods are grouped together in the same class.

3.2.12 Key Design Decisions

1. **Inheritance for User Types:** Patient and Clinician inherit from User to share authentication logic while maintaining role-specific behaviors.
2. **ScreeningCase as Central Entity:** Acts as aggregate root, coordinating between Image, Prediction, and ClinicianFeedback.
3. **Separation of Image and Prediction:** Allows image reuse if reprocessed with different model versions.
4. **Stateless Service:** ModelInferenceService has no state, enabling easy horizontal scaling and deployment across multiple edge devices.
5. **Explicit Feedback Class:** Separates feedback data from Prediction, supporting multiple reviews per case and audit trails.

3.3 Data Design

The SkinSense system uses a relational database (PostgreSQL) to store all application data including user accounts, screening cases, AI predictions, clinician feedback, and audit logs.

3.3.1 Database Design Principles

The database schema is designed following these principles:

- **Normalization:** Tables are normalized to 3rd Normal Form (3NF) to eliminate redundancy
- **Data Integrity:** Foreign key constraints ensure referential integrity
- **Security:** Sensitive fields (passwords, images) are encrypted
- **Auditability:** All critical operations are timestamped and logged
- **Scalability:** Indexed fields enable efficient queries on large datasets

3.3.2 Entity-Relationship Diagram

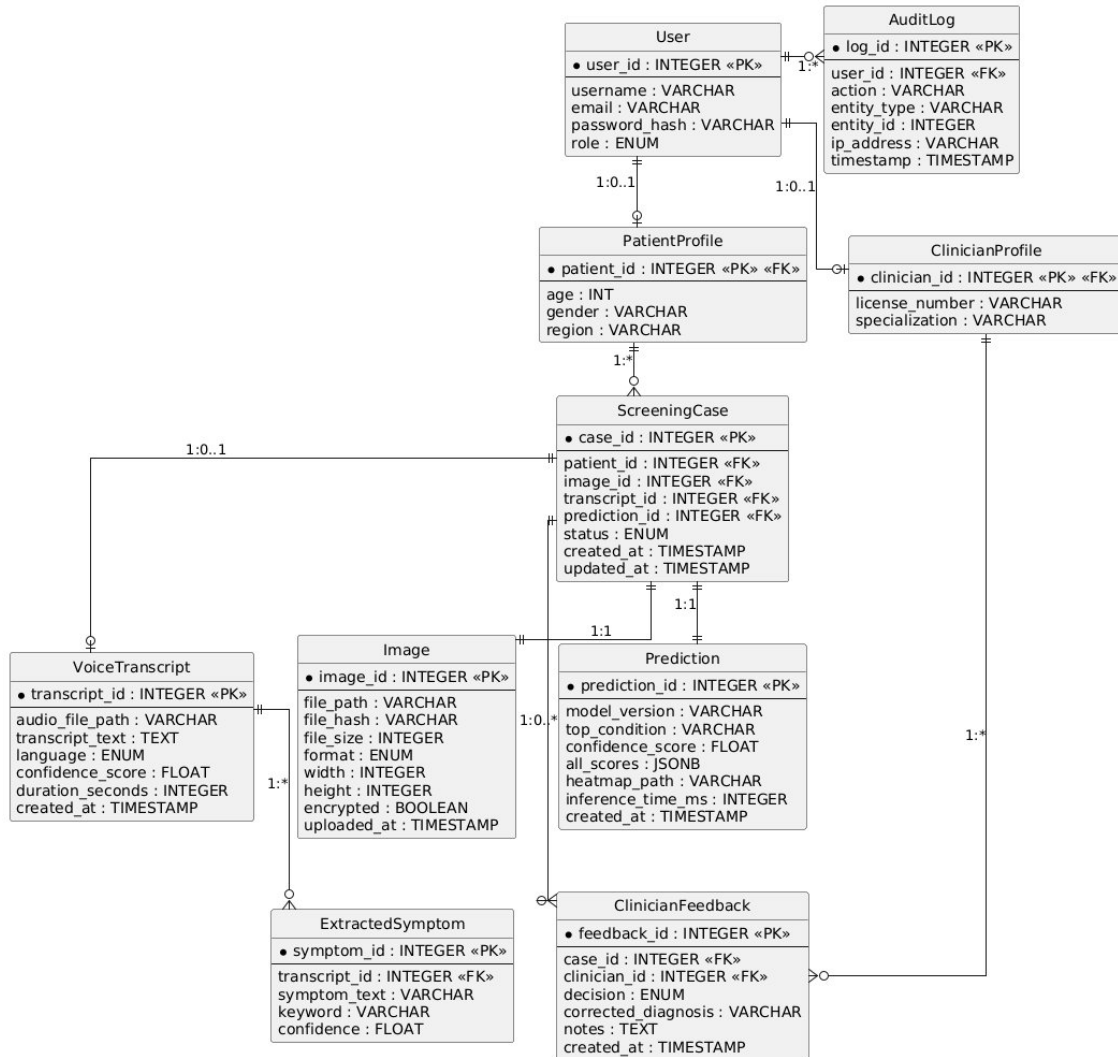


Figure 3.6: Entity-Relationship Diagram for SkinSense Database

3.3.3 Entity Descriptions

The database consists of 10 entities that manage users, patient profiles, screening cases, images, predictions, voice transcripts, symptoms, clinician feedback, and audit logs.

3.3.3.1 User Entity

Central entity for all system users with authentication credentials.

Attributes:

- user_id: INTEGER (Primary Key)
- username: VARCHAR
- email: VARCHAR
- password_hash: VARCHAR (encrypted)
- role: ENUM (patient/clinician)

Relationships:

- 1:0..1 with PatientProfile
- 1:0..1 with ClinicianProfile
- 1:* with AuditLog

3.3.3.2 PatientProfile Entity

Stores patient-specific information.

Attributes:

- patient_id: INTEGER (Primary Key, Foreign Key)
- age: INT
- gender: VARCHAR
- region: VARCHAR

Relationships:

- 1:* with ScreeningCase (patient creates multiple cases)

3.3.3.3 ClinicianProfile Entity

Stores clinician professional information.

Attributes:

- clinician_id: INTEGER (Primary Key, Foreign Key)
- license_number: VARCHAR

- specialization: VARCHAR

Relationships:

- 1:* with ClinicianFeedback

3.3.3.4 ScreeningCase Entity

Central entity linking patients to their screening sessions.

Attributes:

- case_id: INTEGER (Primary Key)
- patient_id: INTEGER (Foreign Key)
- image_id: INTEGER (Foreign Key)
- transcript_id: INTEGER (Foreign Key)
- prediction_id: INTEGER (Foreign Key)
- status: ENUM (pending/reviewed)
- created_at: TIMESTAMP
- updated_at: TIMESTAMP

Relationships:

- 1:1 with Image
- 1:0..1 with VoiceTranscript (optional)
- 1:1 with Prediction
- 1:0..* with ClinicianFeedback

3.3.3.5 Image Entity

Stores dermatological image metadata.

Attributes:

- image_id: INTEGER (Primary Key)

- file_path: VARCHAR
- file_hash: VARCHAR
- file_size: INTEGER
- format: ENUM (JPEG/PNG)
- width: INTEGER
- height: INTEGER
- encrypted: BOOLEAN
- uploaded_at: TIMESTAMP

3.3.3.6 VoiceTranscript Entity

Stores voice recordings and transcriptions.

Attributes:

- transcript_id: INTEGER (Primary Key)
- audio_file_path: VARCHAR
- transcript_text: TEXT
- language: ENUM (ur/en)
- confidence_score: FLOAT
- duration_seconds: INTEGER
- created_at: TIMESTAMP

Relationships:

- 1:* with ExtractedSymptom

3.3.3.7 Prediction Entity

Stores AI model predictions and confidence scores.

Attributes:

- prediction_id: INTEGER (Primary Key)

- `model_version`: VARCHAR
- `top_condition`: VARCHAR
- `confidence_score`: FLOAT
- `all_scores`: JSONB
- `heatmap_path`: VARCHAR
- `inference_time_ms`: INTEGER
- `created_at`: TIMESTAMP

3.3.3.8 ExtractedSymptom Entity

Stores symptom keywords extracted from voice transcripts.

Attributes:

- `symptom_id`: INTEGER (Primary Key)
- `transcript_id`: INTEGER (Foreign Key)
- `symptom_text`: VARCHAR
- `keyword`: VARCHAR
- `confidence`: FLOAT

3.3.3.9 ClinicianFeedback Entity

Stores clinician review and feedback.

Attributes:

- `feedback_id`: INTEGER (Primary Key)
- `case_id`: INTEGER (Foreign Key)
- `clinician_id`: INTEGER (Foreign Key)
- `decision`: ENUM (accept/dispute/correct)
- `corrected_diagnosis`: VARCHAR
- `notes`: TEXT
- `created_at`: TIMESTAMP

3.3.3.10 AuditLog Entity

Comprehensive audit trail for system actions.

Attributes:

- log_id: INTEGER (Primary Key)
- user_id: INTEGER (Foreign Key)
- action: VARCHAR
- entity_type: VARCHAR
- entity_id: INTEGER
- ip_address: VARCHAR
- timestamp: TIMESTAMP

3.3.4 Relationship Cardinalities

Table 3.3: Entity Relationships and Cardinalities

Relationship	Cardinality	Description
User — PatientProfile	1:0..1	User may have one patient profile
User — ClinicianProfile	1:0..1	User may have one clinician profile
PatientProfile — ScreeningCase	1:*	Patient can have multiple cases
ScreeningCase — Image	1:1	Each case has exactly one image
ScreeningCase — VoiceTranscript	1:0..1	Case may have optional transcript
ScreeningCase — Prediction	1:1	Each case has one prediction
VoiceTranscript — ExtractedSymptom	1:*	Transcript has multiple symptoms
ScreeningCase — ClinicianFeedback	1:0..*	Case may have multiple feedbacks
ClinicianProfile — ClinicianFeedback	1:*	Clinician provides multiple feedbacks
User — AuditLog	1:*	User has multiple audit log entries

3.3.5 Database Indexes for Performance

Table 3.4: Database Indexes

Table	Indexed Columns	Purpose
User	username, email	Fast user lookup
ScreeningCase	patient_id, created_at	Patient history retrieval
ScreeningCase	status	Filter pending/reviewed cases
Prediction	model_version, created_at	Model performance tracking
ClinicianFeedback	case_id, clinician_id	Fast feedback retrieval
AuditLog	user_id, timestamp	User activity tracking

3.3.6 Data Security Measures

1. **Password Storage:** Passwords hashed
2. **Image Encryption:** All images encrypted before storage
3. **Data at Rest:** Database configured with encryption.
4. **Data in Transit:** All connections use TLS 1.3
5. **Audit Logging:** All data access logged with timestamps and/or IP addresses

Bibliography

- [1] Aleena Bilal, Aimal Rextin, and Asad Ali. Roman-txt: Forms and functions of roman urdu texting. In *Proceedings of the 19th International Conference on Human-Computer Interaction with Mobile Devices and Services*, pages 1–9, 2017.
- [2] Eric J. Topol. *Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again*. Basic Books, New York, 2019.
- [3] Shehla Zaidi, Abdul Momin Kazi, Atif Riaz, Asad Ali, Rawaha Najmi, Rubina Jabeen, Urooj Khudadad, and Saleem Sayani. Operability, usefulness, and task-technology fit of an mhealth app for delivering primary health care services by community health workers in underserved areas of pakistan and afghanistan: qualitative study. *Journal of Medical Internet Research*, 22(9):e18414, 2020.